Git, GitHub & Vercel

Version Control, Collaboration & Deployment

Track your code, work with a team, and put your projects live

Commits, branches, pull requests, CI, deployments, and a full project workflow

Git GitHub Vercel

OVERVIEW

How the Three Tools Fit Together

Every modern developer uses these three tools together. **Git** records the history of your code on your computer, **GitHub** stores that history online so you can back it up and collaborate, and **Vercel** automatically publishes your project to the internet every time you push to GitHub.

Tool	What it is	Where it lives	Analogy
Git	Version control system	Your computer	A save-game system with unlimited undo
GitHub	Code hosting and collaboration platform	The cloud (github.com)	Google Drive for code, plus teamwork tools
Vercel	Deployment and hosting platform	The cloud (vercel.com)	An automatic publisher that puts your site live

The professional workflow

Step	What you do	Tool
1	Write code and save snapshots (commits) on a branch	Git
2	Push the branch to the online repository	Git + GitHub
3	Open a Pull Request so others can review	GitHub
4	Vercel builds a live **preview link** for the Pull Request	Vercel
5	Merge the Pull Request into the main branch	GitHub
6	Vercel automatically deploys the update to your live site	Vercel

Why this matters

- **Safety:** undo any mistake and return to any earlier version
- **Teamwork:** many people can work on the same project without overwriting each other
- **Portfolio:** your GitHub profile and live Vercel links are what employers look at
- **Speed:** deployment becomes automatic; just push your code

Prerequisite: A computer with a terminal (Command Prompt, PowerShell, Terminal), a code editor like VS Code, and a free GitHub account. Vercel offers a free plan for personal projects.

1 Git Basics

Install, configure, and save your first snapshots

Install and configure (once)

Download Git from git-scm.com, then tell Git who you are. Check the install with `git --version`.

```
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --list                       # check your settings
```

The three areas of Git

Area	Meaning	Command to move changes
Working directory	Your files as you edit them	(edit files)
Staging area	Changes selected for the next snapshot	git add
Repository	The permanent saved history (commits)	git commit

Your first repository

```
mkdir my-project && cd my-project
git init                                # start tracking this folder
echo "# My Project" > README.md

git status                              # see what changed
git add README.md                       # stage one file   (git add . = all)
git commit -m "Add README"              # save the snapshot

git log --oneline                       # view history
```

Everyday commands

Command	Purpose
git status	Show changed, staged, and untracked files
git diff	Show unstaged changes line by line
git diff --staged	Show what will go into the next commit
git log --oneline --graph	Compact history with a branch graph
git restore file.txt	Discard unsaved changes to a file
git restore --staged file.txt	Unstage a file (keep the changes)
git commit --amend	Fix the last commit message or add forgotten files

Ignore files you should not track

Create a file named `.gitignore` in the project root:

```
node_modules/
.env
dist/
*.log
.DS_Store
```

> **Good commit messages:** short, present tense, and specific. Write *"Add login form validation"*, not *"changes"* or *"stuff"*.

2 Branches, Merging & Undoing

Work safely on features without breaking main

What is a branch?

A branch is an independent line of work. The **main** branch holds your stable code, and you create a new branch for every feature or bug fix, then merge it back when it is ready.

```
git branch                          # list branches (* = current)
git switch -c feature/login         # create AND move to a new branch
# ...edit files, add, commit...
git switch main                     # go back to main
git merge feature/login             # bring the feature into main
git branch -d feature/login         # delete the finished branch
```

Resolving a merge conflict

A conflict happens when two branches changed the same lines. Git marks the file; you choose what to keep, remove the markers, then commit:

```
<<<<<<< HEAD
<h1>Welcome back!</h1>              <- your current branch
=======
<h1>Hello, user!</h1>               <- the incoming branch
>>>>>>> feature/login

git add index.html                  # after editing and removing markers
git commit                          # completes the merge
```

Undoing things safely

Situation	Command	Safe for shared history?
Discard edits to a file	git restore file	Yes (local only)
Undo a commit by adding an opposite commit	git revert <commit>	**Yes** (recommended for pushed work)
Move back, keep your changes	git reset --soft HEAD~1	Local commits only
Move back, delete changes	git reset --hard HEAD~1	**Dangerous**; data is lost
Save unfinished work temporarily	git stash / git stash pop	Yes

> **Golden rule:** Never rewrite (reset, rebase, force-push) commits that other people have already pulled. Use `git revert` instead.

3 GitHub

Back up your code and collaborate online

Connect your local repo to GitHub

Create an empty repository on github.com (click **New**), then run the commands GitHub shows you:

```
git remote add origin https://github.com/YOUR-USERNAME/my-project.git
git branch -M main
git push -u origin main           # first push; later just: git push
```

Authentication

- **HTTPS:** GitHub no longer accepts account passwords for Git. Use a **Personal Access Token** (or let Git Credential Manager sign you in via the browser)
- **SSH (recommended):** generate a key once and add the public key in GitHub Settings > SSH and GPG keys

```
ssh-keygen -t ed25519 -C "you@example.com"
cat ~/.ssh/id_ed25519.pub         # copy this into GitHub
ssh -T git@github.com             # test the connection
```

Syncing commands

Command	Purpose
git clone <url>	Download a full copy of a repository
git fetch	Download new changes without merging them
git pull	Fetch and merge the latest changes into your branch
git push	Upload your commits to GitHub
git remote -v	Show linked remote repositories

The Pull Request (PR) workflow

```
git switch -c feature/navbar
# ...make changes and commit...
git push -u origin feature/navbar
# Then on GitHub: click "Compare & pull request", describe the change,
# request a review, address comments, and press "Merge pull request".

git switch main && git pull        # update your local main afterwards
```

Key GitHub features

Feature	Use
Issues	Track bugs, tasks, and ideas; link them to PRs with "Fixes #12"
Fork	Copy someone else's repo to your account to contribute to open source
README.md	The front page of your repo: describe, install, usage, screenshots
GitHub Pages	Free hosting for static sites straight from a repo
GitHub Actions	Automatic workflows: run tests and checks on every push or PR
Branch protection	Require reviews and passing checks before merging into main
GitHub CLI (gh)	Create repos and PRs from the terminal: gh repo create, gh pr create

GitHub Actions: a simple CI workflow

Save as `.github/workflows/ci.yml` to run your build on every push and PR:

```
name: CI
on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
```

```
      - run: npm ci
      - run: npm run build
```

4 Vercel

Deploy your project live with every push

What Vercel does

Vercel is a cloud platform for hosting front-end and full-stack web apps (React, Next.js, Vue, Svelte, static sites, and more). It connects to your GitHub repository, builds your project, and publishes it on a free `your-app.vercel.app` address with HTTPS.

Deploy from GitHub in 5 steps

Step	Action
1	Sign up at vercel.com using your GitHub account
2	Click **Add New > Project** and choose your GitHub repository
3	Vercel auto-detects the framework (e.g., Vite, Next.js) and fills the build settings
4	Add any environment variables, then click **Deploy**
5	Open your live URL; from now on every push redeploys automatically

Production vs. Preview deployments

Trigger	Result
Push or merge to main	**Production** deployment on your main domain
Push to any other branch	**Preview** deployment with a unique URL
Open a Pull Request	Preview link posted on the PR so reviewers can test it live

Typical build settings (Vite + React)

Setting	Value
Framework preset	Vite (auto-detected)
Build command	npm run build
Output directory	dist
Install command	npm install

Environment variables and secrets

Add keys in **Project Settings > Environment Variables**, never in your code or GitHub. In Vite, variables exposed to the browser must start with `VITE_` (Next.js uses `NEXT_PUBLIC_`). Redeploy after changing them.

```
// reading a variable in a Vite app
const api = import.meta.env.VITE_API_URL;
```

React Router fix: single-page app rewrites

If refreshing a page like `/about` shows a 404, add a `vercel.json` in the project root:

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

Vercel CLI (optional)

```
npm install -g vercel
vercel login
vercel                 # deploy a preview
vercel --prod          # deploy to production
vercel env pull        # download env variables into a local .env file
```

More Vercel features

- **Custom domains:** add yours under Project Settings > Domains and update DNS as instructed
- **Serverless Functions:** files in an `api/` folder become backend endpoints
- **Instant rollback:** promote any earlier deployment from the Deployments tab
- **Logs and analytics:** view build errors and runtime logs in the dashboard
- Check vercel.com/docs for current plan limits and features

PUTTING IT ALL TOGETHER

Mini Project: Ship a React App

Create a project, track it with Git, publish it to GitHub, and deploy it on Vercel with a full pull request workflow.

Part A: Create, commit, and push

```
npm create vite@latest my-portfolio -- --template react
cd my-portfolio
npm install
npm run dev                          # test locally, then stop with Ctrl+C

git init
git add .
git commit -m "Initial commit: Vite React app"

# create an empty repo named my-portfolio on github.com, then:
git remote add origin git@github.com:YOUR-USERNAME/my-portfolio.git
git branch -M main
git push -u origin main
```

Part B: Deploy on Vercel

- Go to vercel.com/new, import **my-portfolio**, and click **Deploy**
- Open the live `.vercel.app` link and share it

Part C: Make an update the professional way

```
git switch -c feature/hero-section
# edit src/App.jsx ...
git add .
git commit -m "Add hero section"
git push -u origin feature/hero-section

# 1. On GitHub, open a Pull Request
# 2. Vercel comments on the PR with a Preview URL; test it
# 3. Merge the PR
# 4. Vercel redeploys the production site automatically

git switch main
git pull
git branch -d feature/hero-section
```

Ideas to extend it

- Add the GitHub Actions workflow to build on every PR
- Connect a custom domain on Vercel
- Add a screenshot and live link to your README.md
- Pin this repository on your GitHub profile

TROUBLESHOOTING

Common Problems and Fixes

Problem	Cause	Fix
fatal: not a git repository	You are not inside a repo folder	cd into the project, or run git init
rejected: non-fast-forward (push fails)	GitHub has commits you do not have	git pull, resolve conflicts, then git push
Permission denied (publickey)	SSH key not added to GitHub	Add your public key, then run ssh -T git@github.com
Authentication failed (HTTPS)	Password used instead of a token	Use a Personal Access Token or switch to SSH
Merge conflict	Same lines edited on two branches	Edit the file, remove markers, git add, git commit
Committed a file by mistake	Missing from .gitignore	git rm --cached file, add to .gitignore, commit
Committed a secret (API key)	Key was in the code	**Revoke and replace the key immediately**; removing it later does not erase history
Vercel build fails	Missing dependency, wrong build command, or env variable	Read the build log; run npm run build locally to reproduce
404 on page refresh (Vercel)	Single-page app routing	Add the vercel.json rewrite shown earlier
Env variable is undefined	Missing VITE_ prefix or no redeploy	Rename with the prefix and redeploy

Security warning: Never push passwords, API keys, or `.env` files to GitHub, even in a private repo. Once a secret is committed, treat it as exposed and rotate it.

NEXT STEPS

Learning Roadmap & Cheat Sheet

Stage	Focus	Practice
1. Git basics	init, add, commit, log, .gitignore	Track a small project
2. Branching	branch, switch, merge, resolve conflicts	Add a feature on a branch
3. GitHub	remote, push, pull, SSH, README, issues	Publish 3 projects
4. Collaboration	Forks, pull requests, code review	Contribute to an open-source repo
5. Deployment	Vercel import, preview deploys, env variables	Deploy your portfolio
6. Automation	GitHub Actions, branch protection, custom domain	CI that tests every PR
7. Advanced Git	rebase, cherry-pick, tags, git bisect	Clean history and releases

Quick cheat sheet

Task	Command
Start / copy a repo	git init git clone <url>
Save a snapshot	git add . && git commit -m "message"
New branch	git switch -c name
Merge a branch	git switch main && git merge name
Send / get changes	git push git pull
Temporarily shelve work	git stash git stash pop
Undo safely	git revert <commit>
Deploy (CLI)	vercel vercel --prod

Best practices and resources

- Commit small and often, with clear messages; one logical change per commit
- Never work directly on main for team projects; use branches and pull requests
- Pull before you start work and before you push
- Keep secrets in environment variables, and list `.env` in .gitignore
- Learn more: **git-scm.com/book** (free Pro Git), **docs.github.com**, **vercel.com/docs**, and the interactive **learngitbranching.js.org**

> **Remember:** Every professional project begins with *git init*. Make Git a daily habit and your work will always be safe, shareable, and live.